Ingegneria del Software — Soluzione del Tema 07/06/2023

Esercizio 1

Si consideri la seguente classe `LastHundredPosts` che memorizza i post e i commenti di un nuovo social network, che mantiene solo gli ultimi 100 post di ogni utente e solo i commenti relativi a tali post.

```
public class LastHundredPosts {
  // Ritorna l'insieme degli utenti del social network.
  public /*@ pure @*/ Set<User> getUsers() {...}

  // Ritorna la lista dei post effettuati dall'utente passato per parametro.
  // La lista contiene i post in ordine inverso di inserimento: dal piu' recente al meno recente.
  // Lancia una UnknownUserException se l'utente non e' parte del social network.
  public /*@ pure @*/ List<Post> getPosts(User user) throws UnknownUserException {...}

  // Aggiunge un post avente come autore l'utente passato per parametro. Se l'utente ha gia' effettuato
  // almeno 100 post, cancella il post meno recente tra quelli attualmente salvati per quell'utente.
  // Lancia una UnknownUserException se l'utente non e' parte del social network.
  public void addPost(Post post, User user) throws UnknownUserException {...}

  // Aggiunge un commento al post passato per parametro e avente come autore l'utente passato per parametro.
  // Lancia una UnknownUserException se l'utente non e' parte del social network.
  // Lancia una UnknownPostException se il post non e' presente nel social network.
  public void addComment(Comment comment, Post post, User user)
          throws UnknownUserException, UnknownPostException {...}

  // Ritorna i 10 commenti (di qualunque utente e associati a qualunque post) che hanno ricevuto piu' like.
  // I commenti sono ritornati in ordine inverso di numero di like (prima quelli con piu' like).
  // In caso di parita' l'ordine non e' specificato. In ogni caso, la lista contiene al piu'
  // 10 commenti (o meno, se nel social network non sono presenti almeno 10 commenti).
  public /*@ pure @*/ List<Comment> mostLikedComments() {...}
}

public class Post {
  // Ritorna i commenti associati a questo post.
  public /*@ pure @*/ List<Comment> getComments() {...}
... }

public class Comment {
  // Ritorna il numero di like ricevuti da questo commento.
  public /*@ pure @*/ int numLikes() {...}
... }
```

Domanda a)

Specificare in JML il metodo `addPost`.

Soluzione

Usiamo come soli metodi di base per specificare tutti gli altri `getUsers` e `getPosts`.

Occorre specificare che l'insieme degli utenti non cambia e i post degli altri utenti non cambiano (e non cambiano posizione). Il post viene inserito in prima posizione tra quelli dell'autore e tutti gli altri post si spostano di una posizione. Se la lista conteneva 100 elementi, l'ultimo elemento viene eliminato. In caso di eccezione, l'elenco degli utenti e dei post non cambia.

```
//@requires post!=null && user!=null
//@
//@ensures \old(getUsers()).contains(user) && getUsers().size()==\old(getUsers()).size() &&
//@ \old(getUsers()).containsAll(getUsers()) &&
//@ (\forall User u; getUsers().contains(u) && !u.equals(user);
//@    getPosts(u).size()==\old(getPosts(u)).size() &&
//@    (\forall int i; i>0 && i<getPosts(u).size();
//@       getPosts(u).get(i)==\old(getPosts(u)).get(i) ) ) &&
//@ getPosts(user).size()==(\old(getPosts(user)).size()==100) ? 100 : \old(getPosts(user)).size()+1 &&
//@ getPosts(user).get(0)==post &&
//@ (\forall int i; i>0 && i<getPosts(user).size();
//@    getPosts(user).get(i)==\old(getPosts(user)).get(i-1) )
//@
//@signals(UnknownUserException) !\old(getUsers()).contains(user) &&
//@ getUsers().size()==\old(getUsers()).size() && \old(getUsers()).containsAll(getUsers()) &&
//@ (\forall User u; getUsers().contains(u);
//@    getPosts(u).size()==\old(getPosts(u)).size() &&
```

```
//@    (\forall int i; i>0 && i<getPosts(u).size();
//@       getPosts(u).get(i)==\old(getPosts(u)).get(i) ) ) &&
public void addPost(Post post, User user) throws UnknownUserException {...}
```

Domanda b)

Specificare in JML il metodo `mostLikedComments`.

Soluzione

Il metodo è puro, quindi non occorre specificare che nulla cambia durante la sua esecuzione e non è necessario usare `old` per riferirsi allo stato precedente all'esecuzione del metodo.

Occorre specificare che tutti i commenti ritornati sono commenti relativi a qualche post presente nel social network. Se esistono almeno 10 commenti, allora la lista ritornata ha dimensione 10, altrimenti la lista contiene tutti i commenti presenti nel social network. I commenti sono in ordine inverso di numero di like. Per ogni posizione i della lista ritornata, non devono esistere commenti con un numero di like più alto che sono presenti nel social network ma non sono presenti nella lista in una posizione precedente a i. La lista ritornata non deve contenere duplicati.

Definiamo `numComments` come il numero totale di commenti presenti nel social network. Definiamo `commentInSocial` come un predicato cha vale `true` se e solo se un commento è contenuto nel social network (ovvero è il commento di qualche post presente nel social network).

```
numComments == (\sum User u; getUsers().contains(u);
  (\sum Post p; getPosts(u).contains(p); p.getComments().size()) )

commentInSocial(Comment c) <==> (\exists User u; getUsers().contains(u);
  (\exist Post p; getPosts(u).contains(p); p.getComments().contains(c) ) )

//@ensures (\forall Comment c; \result.contains(c); commentInSocial(c) &&
//@ \result.size() == (numComments<10) ? numComments : 10 &&
//@ (\forall int i; i>0 && i<\result.size();
//@    \result.get(i).numLikes()<=\result.get(i-1).numLikes() &&
//@    (\forall Comment c; commentInSocial(c) && c.numLikes()>\result.get(i).numLikes();
//@       \result.indexOf(c)>=0 && \result.indexOf(c)<i ) ) &&
//@ (\forall int i; i>=0 && i<\result.size()-1;
//@    (\forall int j; j>i && j<\result.size();
//@       ! \result.get(i).equals(\result.get(j)) ) )
public /*@ pure @*/ List<Comment> mostLikedComments() {...}
```

Domanda c)

Si consideri una implementazione della classe `LastHundredPosts` che utilizza una mappa per memorizzare i dati di ogni utente. Per ogni utente, la mappa contiene una lista con tutti i post di quell'utente.

```
public class LastHundredPosts {
  private Map<User, List<Post>> posts;
  ...
}
```

Per questa implementazione scrivere l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione deve dire che la mappa non è nulla, che non contiene chiavi o valori nulli, che nessuna lista contiene valori nulli o ripetuti, che nessuna lista contiene più di 100 post.

```
//@private invariant
//@ posts!=null && !posts.containsKey(null) && !posts.containsValue(null) &&
//@ (\forall User u; posts.containsKey(u); !posts.get(u).contains(null) &&
//@    posts.get(u).size()<=100 && (\forall int i; i>0 && i<posts.get(u).size();
//@       (\forall int j; j>=0 && j<i; !posts.get(u).get(i).equals(post.get(u).get(j)) ) ) )
```

La funzione di astrazione "mappa" gli attributi privati sui metodi base usati per la specifica: `getUsers` e `getPosts`.

```
//@private invariant
//@ getUsers().size()==posts.keySet().size() && getUsers().containsAll(posts.keySet()) &&
//@ (\forall User u; posts.containsKey(u); getPosts(u).size()==posts.get(u).size() &&
//@    (\forall int i; i>=0 && i<getPosts(u).size(); getPosts(u).get(i)==posts.get(u).get(i)) )
```

Domanda d)

Si consideri una classe `LastHundredPostsOrdered` che ridefinisce il metodo `mostLikedComments` per fare in modo che, in caso di parità nel numero di like di due commenti, sia sempre ritornato prima il commento più recente.

`LastHundredPostsOrdered` può essere definita come sottoclasse di `LastHundredPosts` senza violare il principio di sostituzione di Liskov? Viceversa, `LastHundredPosts` può essere definita come sottoclasse di `LastHundredPostsOrdered`?

Soluzione

La classe `LastHundredPostsOrdered` può essere definita come sottoclasse di `LastHundredPosts` in quanto aggiunge un vincolo (rafforza) la post-condizione del metodo `mostLikedComments`, senza alterare le condizioni di altri metodi o le proprietà invarianti ed evolutive della classe. Viceversa, `LastHundredPosts` non può essere definita come sottoclasse di `LastHundredPostsOrdered` in quanto, togliendo l'ordine nei commenti, indebolirebbe la post-condizione del metodo `mostLikedComments`.

Esercizio 2

Si consideri la seguente classe `DataBuffer` che contiene dati numerici che possono essere scritti e letti a partire da una particolare posizione detta "cursore".

```
public class DataBuffer {
  private double[] data;
  private int cursor;

  public DataBuffer() { data = new double[100]; cursor = 0; }

  public double getDataAtCursor() {
    synchronized(data) { return data[cursor]; }
  }

  public void setDataAtCursor(double d) {
    synchronized(data) { data[cursor] = d; }
  }

  public void setCursorPos(int pos) {
    synchronized(cursor) { cursor = pos; }
  }
}
```

Domanda a)

Il codice è correttamente sincronizzato? Motivare la propria risposta e in caso di risposta negativa modificare un solo metodo della classe affinché la sincronizzazione risulti corretta.

Soluzione

I metodi che accedono all'attributo `cursor` acquisiscono lock su oggetti diversi. Inoltre, il metodo `setCursorPos` acquisisce il lock su un oggetto di un tipo base (`int`), il che non è consentito dal linguaggio. È possibile risolvere entrambi i problemi modificando il metodo come segue, garantendo che tutti i metodi acquisiscano il lock sul medesimo oggetto `data`:

```
public void setCursorPos(int pos) {
  synchronized(data) { cursor = pos; }
}
```

Domanda b)

Si modifichi il metodo `getDataAtCursor` in modo da sospendere il chiamante se il dato da ritornare risultasse minore di zero. Il chiamante verrà sbloccato quando il dato diventerà positivo.

Soluzione

```
public double getDataAtCursor1() throws InterruptedException {
  synchronized(data) {
    while(data[cursor]<0) data.wait();
    return data[cursor];
  }
}
```

Si devono anche modificare i metodi `setDataAtCursor` e `setCursorPos` come segue:

```
public void setDataAtCursor(double d) {
  synchronized(data) {
    data[cursor] = d;
    if(d>=0) data.notifyAll();
  }
}

public void setCursorPos(int pos) {
  synchronized(data) {
    cursor = pos;
    if(data[pos]>=0) data.notifyAll();
  }
}
```

Domanda c)

Si consideri il seguente metodo della classe `DataBuffer` che crea due thread e li esegue:

```
public void change() {
  new Thread(() -> {
    for(int i=0; i<100; i++) {
      setCursorPos(i);
      setDataAtCursor(i);
    }
  }).start();

  new Thread(() -> {
    for(int i=0; i<100; i++) {
      setCursorPos(i);
      setDataAtCursor(i*2);
    }
  }).start();
}
```

È corretto affermare che al termine dell'esecuzione dei due thread il contenuto dell'array `data` contiene in ogni posizione `i` il valore `i` oppure il valore `i*2`? Motivare la propria risposta e in caso di risposta negativa modificare i thread affinché la proprietà sia rispettata.

Soluzione

La proprietà non è rispettata perché uno dei due thread potrebbe impostare il cursore a una certa posizione per poi, prima di impostare il dato nella posizione del cursore, cedere il controllo all'altro thread che potrebbe a sua volta impostare il dato con un valore che non sarà più "sincronizzato" con la posizione del cursore. Il codice che segue risolve il problema:

```
public void change() {
  new Thread(() -> {
    for(int i=0; i<100; i++) {
      synchronized(data) {
        setCursorPos(i);
        setDataAtCursor(i);
      }
    }
  }).start();

  new Thread(() -> {
    for(int i=0; i<100; i++) {
      synchronized(data) {
        setCursorPos(i);
        setDataAtCursor(i*2);
      }
    }
  }).start();
}
```

Esercizio 3

Si consideri il seguente metodo:

```
1  public static int doSomething(int x, int y) {
2    if (x>=0 && y>1) {
3      while (y>0) {
4        if (y<3) {
5          x+=y;
6        }
7        if (y>3 && x>0) {
8          x--;
9        }
10       y--;
11     }
12   }
13   return x;
14 }
```

Domanda a)

Si determini un insieme minimo di casi di test per garantire la copertura delle istruzioni (statement coverage).

Soluzione

Il caso (1,4) è sufficiente: il loop viene eseguito quattro volte, alla prima iterazione copre l'istruzione 8, nelle terza (e nella quarta) l'istruzione 5.

Domanda b)

Si determini un insieme minimo di test per garantire la copertura delle decisioni (edge coverage).

Soluzione

Il caso precedente, durante l'esecuzione di diverse iterazioni del loop, copre, oltre alle decisioni vere degli if interni al ciclo, anche le decisioni false dei medesimi if. Alla prima iterazione copre il ramo else "nascosto" del primo if interno, alla seconda iterazione il ramo else "nascosto" del secondo if interno.

Manca soltanto la copertura della decisione false dell'if esterno al loop: basta aggiungere un caso di test, ad esempio (1, 1).

Domanda c)

Si determini un insieme minimo di test per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Usando i casi precedenti manca ancora la copertura di due condizioni:

Nel primo if, la condizione per cui $x >= 0$ è falsa, quindi ad esempio (-1, qualunque).

Nel terzo if, la condizione per cui $y > 3$ è vera con $x > 0$ è falsa: sostituendo il caso (1, 4) con il caso (1, 5) si riesce a coprire anche questa condizione.

In tutto, sono quindi sufficienti 3 casi: (-1, 0), (1, 1), (1, 5).

Esercizio 4

Si considerino le seguenti dichiarazioni di classi Java.

```
abstract class Document {
  public abstract void addTo(Channel c);
  public void addTo(PublicChannel c) { System.out.println("Doc added to " + c); }
}
class Music extends Document {
  public void addTo(Channel c) { System.out.println("Music added to " + c); }
}
class Podcast extends Document {
  public void addTo(Channel c) { System.out.println("Podcast added to " + c); }
  public void addTo(PublicChannel c) { System.out.println("Public Podcast added to " + c); }
}
class Channel {
  public void insert(Document p) { p.addTo(this); }
  public String toString() { return "a channel"; }
}
class PublicChannel extends Channel {
  public String toString() { return "public Channel"; }
}
```

```
1  class Main {
2          public static void main(String[] args) {
3                  Vehicle car = new Car();
4                  HybridCar hybridCar = new HybridCar();
5                  Car electricCar = new ElectricCar();
6
7                  car.start();
8                  car.start(true);
9                  car.start(false);
10
11                 hybridCar.start();
12                 hybridCar.start(true);
13                 hybridCar.start(false);
14
15                 electricCar.start();
16                 electricCar.start(true);
17                 electricCar.start(false);
18
19
20         }
21 }
```

Domanda a)

Ci sono linee del programma non valide? In caso affermativo indicarle, motivando brevemente la risposta e precisando se causano errori a tempo di compilazione o di esecuzione.

Soluzione

Riga 3 genera errore di compilazione (non si può istanziare una classe astratta). Anche riga 7 genera errore di compilazione (Music non è sottoclasse di Podcast). Di conseguenza sono da eliminare le righe che usano le variabili `d1` e `m2`, ossia le righe 11 e 18.

Domanda b)

Eliminate le righe che generano gli errori individuati al punto precedente, indicare cosa stampa il programma, motivando brevemente la risposta. Riportare sempre il numero di riga.

Soluzione

12. Podcast added to a channel

13. Music added to public Channel

14. Podcast added to public Channel

15. Music added to public Channel

16. Podcast added to public Channel
17. Music added to public Channel